



EDE PLATFORM · DEVELOPER GUIDE

Constraints & Validation

How to Enforce a Rule

A practical guide to the three tools for enforcing invariants on a business model — the two **constraints**:* DB declarations, the **@api.constrains** code decorator, and lifecycle **hooks** — with copy-paste examples and a decision guide.

In one line: declare **shape** on the field, a **database-guaranteed** invariant with `constraints.Check / constraints.Unique`, a **business rule** with `@api.constrains`, and anything **op-specific** (delete guards, post-write effects) with `@api.on_hook`.

01 Four tools, one decision

Enforcing a rule on a model is a question of **where** it should run. Pick the highest, cheapest tool that can express your rule.

TOOL 1

Field flags

Is the shape valid?

required=True

unique=True (single field)

Char(max_length, regex)

Reference(on_delete)

TOOL 2 · DB-DRIVEN

constraints.*

A guarantee the database enforces against ANY writer.

constraints.Check("min_qty <= max_qty")

constraints.Unique("code","org_id")

TOOL 3 · CODE-DRIVEN

@api.constrains

A business rule — conditional, cross-field, cross-record.

@api.constrains("stage","partner_id")

→

raise ⇒ veto

TOOL 4 · LIFECYCLE

@api.on_hook

Op-specific: delete guards, post-write effects, raw cmd access.

@api.on_hook("pre.crm.lead.delete")

·

post.{model}.{op}

Which one do I reach for?

You need to...	Use	Runs
Require presence, single-field uniqueness, string length/pattern, FK behaviour	Field flags — <code>fields.X(required=, unique=, ...)</code>	DB + load
Guarantee a predicate over one row's columns (e.g. <code>min ≤ max</code>) that must hold against raw SQL too	<code>constraints.Check</code>	Database
Guarantee composite uniqueness across 2+ columns	<code>constraints.Unique</code>	Database
Enforce a business rule : conditional, cross-field, or needing a lookup — with a friendly message	<code>@api.constrains</code>	App, pre-write
Guard a specific operation (block delete), run a post-write side effect, or read the raw <code>cmd</code>	<code>@api.on_hook</code>	App, around op

The rule of thumb
Field flags for shape → **constraints.*** when the database itself must guarantee it → **@api.constrains** for a business rule with a friendly message → **@api.on_hook** when you need a specific operation, the post phase, or the raw command. The next four sections cover each in turn.

02 DB-driven — the `constraints.*` namespace

Import `constraints` exactly like `fields`, then declare table constraints as class-body attributes beside the fields they govern. The database enforces them against **every** writer — the ORM, the data loader, raw SQL.

CONSTRAINTS.CHECK — A SQL PREDICATE OVER THE ROW'S OWN COLUMNS

```
from ede.core.kernel import fields
from ede.core.kernel import constraints

@api.model("logistics.product")
class Product(DomainModel):
    min_qty = fields.Integer(default=0)
    max_qty = fields.Integer(default=0)

    # the attribute name (qty_bounds) becomes the DB constraint name
    qty_bounds = constraints.Check("min_qty <= max_qty", message="min_qty must be ≤ max_qty")
```

CONSTRAINTS.UNIQUE — COMPOSITE UNIQUENESS (USE FIELDS.X(UNIQUE=TRUE) FOR A SINGLE COLUMN)

```
code = fields.Char()
org_id = fields.Reference("res.organization")

code_per_org = constraints.Unique("code", "org_id", message="code must be unique per organization")
```

Yes — this is a composite key: the *combination* is unique

`Unique("code", "org_id")` constrains the **pair** `(code, org_id)` as a whole — **not** each column on its own. The same `code` may appear under different organizations, and one organization may hold many codes; only the exact `(code, org_id)` combination cannot repeat.

#	code	org_id	Result
1	PROD-A	Org 1	INSERTED first row
2	PROD-A	Org 2	ALLOWED same code, different org
3	PROD-B	Org 1	ALLOWED same org, different code
4	PROD-A	Org 1	REJECTED duplicate <code>(code, org_id)</code> → your message

⚠ Not the same as two single-field uniques

Marking `code` and `org_id` each `unique=True` would (wrongly) forbid the same code in two orgs and reuse of an org — rows 2 and 3 above would fail. Composite `Unique(...)` is the correct tool for "unique **per** org" scoping. Column order doesn't change the guarantee (the set of columns is what's unique); it only affects lookups that reuse the index.

What you get

Aspect	Behaviour
Enforcement	Real DB <code>CHECK</code> / <code>UNIQUE</code> — holds even if a write bypasses the ORM
On violation	Raises <code>ConstraintViolation</code> carrying your <code>message</code> → surfaces as HTTP 422 (a clean error, not a 500)
Naming	The attribute name is the identity → <code>ck_logistics_product_qty_bounds</code> / <code>uq_..._code_per_org</code>
Check input	A SQL expression string only — one row's own columns; no Python, no joins/subqueries

⚠ You must generate a migration

After adding a `constraints.*` declaration, run `ede migrate generate --app <app>` to emit the named constraint. **Never hand-author the migration.** Names are capped to Postgres' 63-char limit automatically; a `Unique` naming a non-existent field fails fast at **boot**.

When a Check isn't enough

A `CHECK` can only see the current row's columns. The moment your rule is conditional, spans records, or needs a lookup, it belongs in `@api.constrains` (next section) — not a `Check`.

03 From your declaration to database DDL

When you run `ede migrate generate`, each `constraints.*` and field declaration compiles to real DDL. Here is exactly what you write and what the database ends up with.

YOU WRITE (PYTHON)

```
code    = fields.Char(required=True)
org_id  = fields.Reference("res.organization")
min_qty = fields.Integer()
max_qty = fields.Integer()

qty_bounds = constraints.Check("min_qty <= max_qty", message="min ≤ max")
code_per_org = constraints.Unique("code", "org_id", message="unique per org")
```

DATABASE GETS (GENERATED DDL — SQL)

```
CREATE TABLE logistics_product (
  dbid          INTEGER          NOT NULL PRIMARY KEY AUTOINCREMENT,
  record_uuid   VARCHAR(36)      NOT NULL,
  code          VARCHAR(255)     NOT NULL,          -- required=True → NOT NULL
  org_id        VARCHAR(36),      -- Reference → uuid FK column
  min_qty       INTEGER,
  max_qty       INTEGER,
  -- ... auto columns: created_at_utc, updated_at_utc, revision, ...

  CONSTRAINT ck_logistics_product_qty_bounds CHECK (min_qty <= max_qty),
  CONSTRAINT uq_logistics_product_code_per_org UNIQUE (code, org_id),
  CONSTRAINT fk_logistics_product_org_id_res_organization
    FOREIGN KEY (org_id) REFERENCES res_organization (record_uuid) ON DELETE RESTRICT
);
```

How each piece maps

You write	Becomes in the database
<code>constraints.Check("min_qty <= max_qty")</code>	<code>CONSTRAINT ck_<table>_qty_bounds CHECK (min_qty <= max_qty)</code>
<code>constraints.Unique("code", "org_id")</code>	<code>CONSTRAINT uq_<table>_code_per_org UNIQUE (code, org_id)</code>
attribute name <code>qty_bounds</code>	constraint name <code>ck_<table>_qty_bounds</code> (capped to 63 chars)
<code>message="..."</code>	nothing — the message is never stored in the DB; it lives in Python and is attached when the DB rejects the write

⚠ Code-driven tools generate NO DDL

`@api.constrains` and `@api.on_hook` add **nothing** to the schema — no column, no constraint, no migration. They live only in Python and run at write time. Only field flags and `constraints.*` change the database.

On an existing table

Adding a constraint later emits an `ALTER TABLE ... ADD CONSTRAINT ...` instead of the inline form above (on SQLite it runs via a table rebuild). Either way you get the same named constraint — always via `ede migrate generate`, never hand-written.

04 Code-driven — `@api.constrains`

For business rules the database can't express. Decorate a method with the **trigger fields**; it runs on **create and update**, receives the record with the incoming payload already merged, and **raises to veto**. One declaration — no separate create/update hooks, no manual merge.

SIGNATURE — (SELF, RECORD); RECORD REFLECTS THE RESULTING STATE

```
@api.constrains("stage", "partner_id")
def _won_needs_partner(self, record) -> None:
    if record.stage == "won" and not record.partner_id:
        raise ValueError("a lead cannot be marked won without a partner")
```

CROSS-RECORD — SELF.ENV IS INJECTED, SO YOU CAN LOOK UP OTHER MODELS

```
@api.constrains("code")
def _unique_active_code(self, record) -> None:
    clash = self.env.models["crm.lead"].search(
        [("code", "=", record.code), ("active", "=", True), ("id", "!=", record.id)])
    if clash:
        raise ValueError(f"code {record.code} already used by an active lead")
```

The contract

Aspect	Behaviour
Runs on	<code>create</code> and <code>update</code> from one declaration
Trigger fields	On update the check is skipped unless the payload touches one of them; on create it always runs. They also document what the rule depends on.
record	Pre-merged: incoming payload value if the field is being written, else the current stored value — so you validate the <i>resulting</i> state, no manual merge
Veto	Raise any exception → aborts the write inside its transaction; the message reaches the caller (HTTP 422)
self.env	Injected — read other models with <code>env.models[...].search(...)</code>

Why not just write two hooks?

`@api.constrains` is sugar over a `pre.create` + `pre.update` hook pair — but it writes the rule once, merges the record for you, and gates on the trigger fields. Reach for raw hooks only when you need the **delete** op, the **post** phase, or the raw `cmd` (next section).

⚠ Pre-write only

`@api.constrains` covers create + update. It does **not** fire on delete and has no post-write phase — those are `@api.on_hook`.

05 Lifecycle hooks — @api.on_hook

The general mechanism underneath everything above. A hook fires **around a specific command** — `pre.{model}.{op}` before, `post.{model}.{op}` after. Pre-hooks raise to veto; post-hooks run side effects after a successful write. Use them when `@api.constrains` is the wrong shape.

DELETE GUARD — AN OP @API.CONSTRAINS CAN'T EXPRESS

```
@api.on_hook("pre.crm.lead.delete")
def _block_won_delete(self, cmd) -> None:
    if cmd.record.stage == "won":
        raise ValueError("won leads are retained for audit - archive instead")
```

POST-WRITE SIDE EFFECT — RUNS ONLY AFTER THE WRITE SUCCEEDS

```
@api.on_hook("post.crm.lead.update")
def _notify_owner(self, cmd) -> None:
    self.env.emit("crm.lead.changed", {"id": cmd.record.id})
```

What a hook gives you that a constraint doesn't

Capability	Detail
Any operation	<code>create</code> · <code>update</code> · <code>delete</code> · any domain command (<code>pre.crm.lead.qualify</code>)
pre and post	<code>pre.*</code> can veto; <code>post.*</code> runs after success (cache busting, events, audit)
Raw <code>cmd</code>	Full command: <code>cmd.record</code> (live record; <code>__new__</code> on pre-create), <code>cmd.payload</code> , <code>self.env</code>
Wildcards	<code>pre.*.delete</code> fires on every model's delete — for cross-cutting concerns
Reads excluded	<code>search</code> / <code>read_one</code> / <code>count</code> / <code>read_group</code> never fire hooks — validation is write-only

⚠ On update, validate the resulting state

Raw hooks hand you `cmd.record` (current values) and `cmd.payload` (the changes) **separately** — merge them before checking, or you'll validate stale data. This merge is exactly the boilerplate `@api.constrains` removes; prefer it for plain create/update rules.

06 Putting it together & gotchas

A single model often uses several tools at once — each layer catching what the one above can't. Here's a model that stacks all four, then the traps to avoid.

ONE MODEL, LAYERED DEFENSES

```
@api.model("logistics.product")
class Product(DomainModel):
    code = fields.Char(required=True)          # shape (field flag)
    org_id = fields.Reference("res.organization")
    min_qty = fields.Integer(default=0)
    max_qty = fields.Integer(default=0)
    stage = fields.Char(default="draft")

    qty_bounds = constraints.Check("min_qty <= max_qty", message="min ≤ max")      # DB
    code_per_org = constraints.Unique("code", "org_id", message="unique per org") # DB

    @api.constrains("stage", "max_qty")                                           # business rule
    def _launch_needs_stock(self, record) -> None:
        if record.stage == "launched" and (record.max_qty or 0) <= 0:
            raise ValueError("a launched product needs a positive max_qty")

    @api.on_hook("pre.logistics.product.delete")                                 # op guard
    def _block_launched_delete(self, cmd) -> None:
        if cmd.record.stage == "launched":
            raise ValueError("archive a launched product, don't delete it")
```

DO

- Give every rule a clear `message=` — it's what the user sees.
- Run `ede migrate generate` after any `constraints.*` change.
- List real trigger fields on `@api.constrains` — they gate + document.
- Read other models with `env.models[...].search()` inside the rule.

DON'T

- Put Python logic or a join in a `Check` — SQL, one row only.
- Expect `@api.constrains` to fire on **delete** — use a hook.
- Check only the payload on update — validate the **merged** state.
- Hand-author a migration for a constraint — always generate it.

Errors surface cleanly everywhere

All four tools raise a plain exception to reject a write. Field/DB constraints and `ConstraintViolation` and any `ValueError` you raise map to an HTTP **422** with your message — the same clean shape whether the rule ran in the database or in Python.

Further reading

Design + rationale: [roadmap/platform/15-declarative-constraints.md](#) . Built-state mirror: [docs/platform-15-declarative-constraints.md](#) . Deep-dive on the hook pipeline: the companion *Hook-Driven Validation in EDE* explainer.